

Week 3 Day 5 — AFL Assistant Executive Report

Product goal.

Ship a domain-locked AFL assistant combining factual Q&A, structured statistics and probabilistic prediction behind explicit routing and validation. Unsupported sports and prompt-injection attempts are refused rather than answered out of scope.

Architecture.

Request → AFL scope/injection gate → router → direct AFL answer / retrieval / prediction / refusal → validation → response. FastAPI exposes **/chat** with message + conversation_id; the optional Streamlit UI uses the same endpoint. Every tool/model call runs under a hard timeout, and requests are rate-limited per client. Prediction outputs use one centralized disclaimer: “Predicted probability, not a certainty.”

Evaluation.

The regression suite contains 30 cases across four categories, generated by running the live pipeline end-to-end (*evaluation/run_eval.py*) rather than hand-authored expectations. The supplied 2025 holdout contains 216 matches. The reproducible match model achieved 65.28% accuracy and 0.4275 macro F1; the naive ladder-position baseline achieved 63.43%. This is a transparent internal benchmark, not a bookmaker/public forecast claim.

Results by category.

Category	Cases	Passes	Pass rate
Factual / retrieval	8	8	100%
Scope guardrail	9	9	100%
Prediction sanity	6	6	100%
Multi-turn coherence	7	7	100%
Total	30	30	100%

Weakest category & improvement.

All categories currently pass at 100% or above, but **Multi-turn coherence** is the weakest by design margin rather than by pass rate: the pipeline is stateless per call (*conversation_id* is logged but not yet used to recall prior turns), so a follow-up like “what about 2023?” only works if it happens to be self-contained. **Concrete improvement:** persist per-conversation turn history (team/season/last intent) keyed by *conversation_id* and have the router resolve pronouns/ellipsis (“that”, “them”, “what about <year>”) against it before falling back to a clarification.

Known limitations.

The supplied data ends on 2025-09-27, so the system does not invent 2026 fixtures. Production requires a live fixture/results source. AFL outcomes are noisy; player availability/selection is not fully modeled. Monitoring therefore focuses on drift and calibration rather than promising perfect forecasts.

Production Hardening, Monitoring & Next Steps

Hardening.

Scope checks run before retrieval or prediction; injection-like requests are refused. Unresolved entities/dates produce clarification instead of guesses. API inputs are bounded, every tool/model call runs under a hard timeout, requests are rate-limited per client IP, and repeated off-topic/injection attempts are counted per conversation and flagged in the logs. Structured logs capture query, intent, tools called, latency, token usage and prediction metadata.

Monitoring.

Track p50/p95 latency, API 5xx rate, tool/model error rate, off-topic leak rate, prediction accuracy and calibration. Starting alerts: p95 >3s for 15 minutes; API 5xx >2% for 15 minutes; tool/model errors >3% daily; any confirmed off-topic leak; or a rolling accuracy drop >5 percentage points versus the previous window.

Weekly refresh loop.

Ingest completed results → validate schema/duplicates → recompute leakage-safe features → run the regression suite → compare against the ladder baseline → retrain only if acceptance checks pass → version model/data/schema → deploy with rollback.

Next steps.

Connect a live fixture/results provider; add confirmed player-selection inputs; move conversation state to durable, cross-turn storage; move rate limiting to a shared store/API-gateway for multi-instance deployments; expand adversarial and calibration evaluation; keep the current model available for rollback.

Demo.

The included 5–7 minute script/deck shows a factual question, dated prediction, off-topic/injection refusal and multi-turn conversation, followed by evaluation and monitoring.